

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА
Институт искусственного интеллекта
Кафедра проблем управления

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине **Программное обеспечение мехатронных и робототехнических систем**

Тема лабораторной работы: «Отладка программного обеспечения робототехнических систем с использованием виртуального моделирования»

Позднева А. И. _____
Студенты группы: КРБО-01-23 Лазарева Д. Г. _____
Котова А. В. _____
Преподаватель: ст. преподаватель Морозов А. А. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

Цель работы: получение навыков моделирования объекта управления в промышленных системах автоматического управления и создание функциональных блоков.

Задание: создать виртуальную систему управления (рис. 1), включающую: модель объекта управления (рис. 2), ПИ-регулятор (рис. 3), сумматор и обратную связь. Передаточная функция объекта:

$$W = \frac{1}{k_e(T_m s+1)}$$

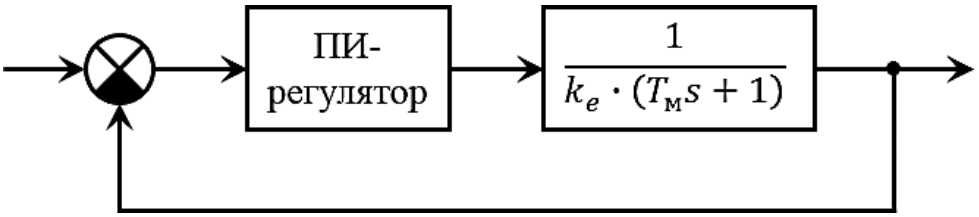
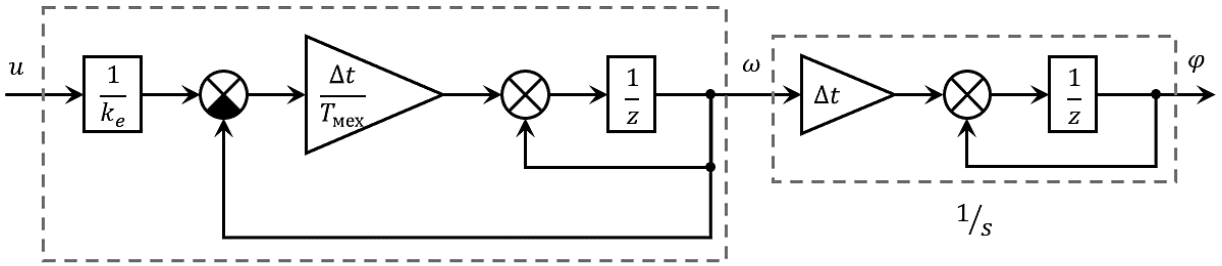


Рис. 1 - Структура системы управления



$$W = \frac{1}{k_e \cdot (T_м s + 1)}$$

Рис. 2 - Структура объекта управления

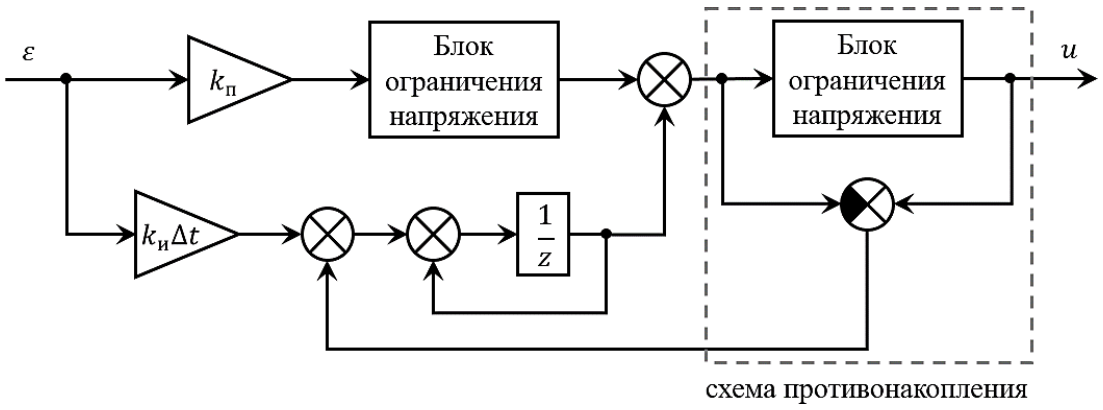


Рис. 3 - Структура ПИ-регулятора

ХОД РАБОТЫ

Лабораторная работа была выполнена в программной среде Automation studio без конфигурации и оборудования, но с объектами ANSI C Program и ANSI C Library Motor Control.

Были созданы 3 функциональных блока - FB_Integrator, FB_Motor, FB_Regulator.

FB_Integrator - выполняет роль модели интегратора. На входе - значения интегрирующего звена, на выходе - соответственное значение с интегрирующего звена. Описание параметров представлено в Таблице 1.

Блок интегратора заменяет непрерывное интегрирование численным методом прямоугольников.

Таблица 1. Параметры функционального блока FB_Integrator

Конфигурация	Имя	Тип данных	Описание
вход	in	REAL	вход интегрирующего звена
выход	out	REAL	выход интегрирующего звена
внутреннее состояние	dt	REAL	шаг расчета [с]
внутреннее состояние	prev_value	VAR	Предыдущее состояние

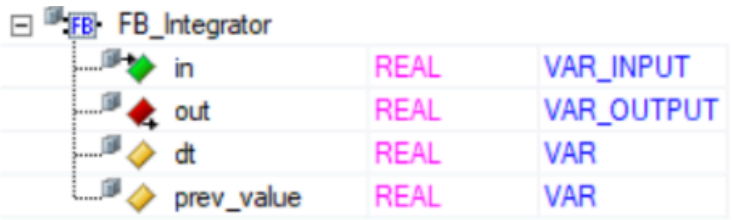


Рис. 4 - Переменные integrator в приложении

Следующий блок, FB_Motor, описывает математическую модель двигателя. Он как бы программирует физику ДПТ на основе дифференциального уравнения динамики вращения. Вход - напряжение, выход - частота вращения и положения. Его параметры представлены в Таблице 2. Расчет на выходе происходит в соответствии со схемой ДПТ на Рис 2.

Таблица 2. Параметры функционального блока FBMotor

Конфигурация	Имя	Тип данных	Описа ние
вход	u	REAL	входное напряжение [В]
выход	w	REAL	частота вращения [об/мин]
выход	phi	REAL	положение [рад]
внутреннее состояние	integrator	FB _Integrator	интегратор
внутреннее состояние	Tm	REAL	электромеханическая постоянная времени [с]
внутреннее состояние	ke	REAL	постоянная ЭДС двигателя [В•мин/об]
внутреннее состояние	dt	REAL	шаг расчета [с]

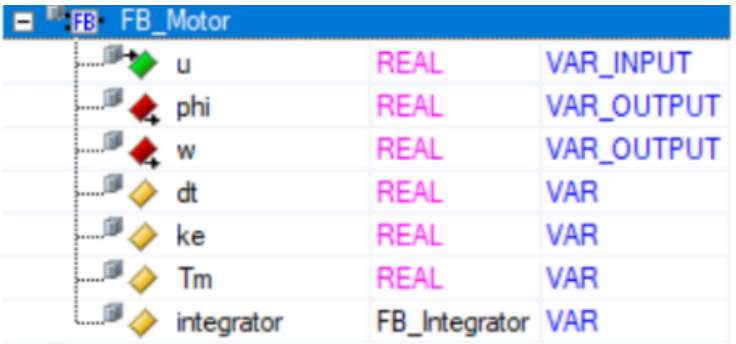


Рис. 5 - Переменные motor в приложении

FB_Regulator - выполняет функции ПИ-регулятора с защитой от интегрального насыщения. На вход подаётся рассогласование между реальным воздействием и реальной скоростью вала ДПТ. На выходе - напряжение. Описание параметров представлено в Таблице 3.

Таблица 3. Параметры функционального блока FB_Regulator

Конфигурация	Имя	Тип данных	Описание
вход	e	REAL	рассогласование между задающим воздействием и реальной скоростью вращения вала ДПТ [об/мин]
выход	u	REAL	напряжение, подаваемое на вход ДПТ [В]
внутреннее состояние	k_p	REAL	пропорциональный коэффициент регулятора
внутреннее состояние	k_i	REAL	интегральный коэффициент регулятора
внутреннее состояние	integrator	FB_Integrator	интегратор
внутреннее состояние	iyOld	REAL	хранение предыдущего значения схемы противонакопления
внутреннее состояние	max_abs_value	REAL	граница блока ограничения [В]
внутреннее состояние	dt	REAL	шаг расчета [с]

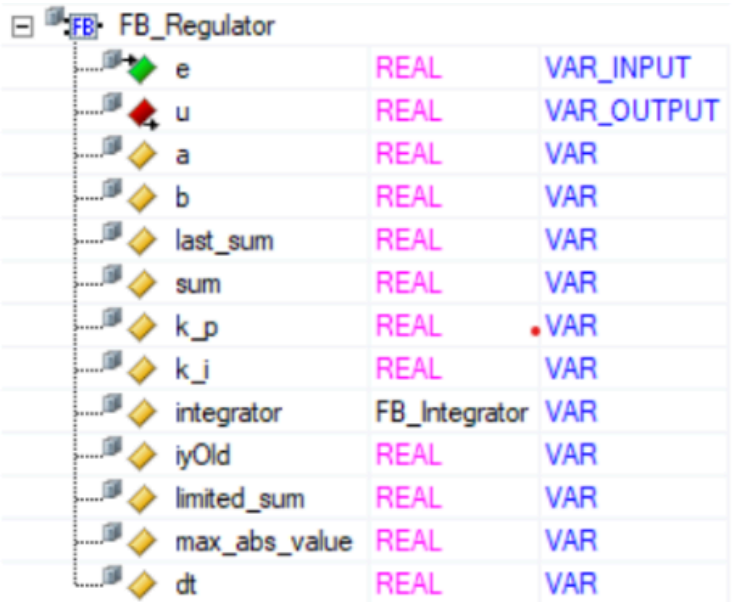


Рис.6 - Переменные regulator в приложении

Затем были созданы экземпляры объектов в глобальных переменных программы.

Name	Type	Reference	Constant	Retain	Replicable
enable	BOOL	☐	☐	☐	☑
regulator	FB_Regulator	☐	☐	☐	☑
e	REAL	☐	☐	☐	☑
counter	USINT	☐	☐	☐	☑
speed	REAL	☐	☐	☐	☑
fb_motor1	FB_Motor	☐	☐	☐	☑
fb_motor2	FB_Motor	☐	☐	☐	☑

Рис.7 - Переменные основной части программы

Расчёт коэффициентов ПИ-регулятора был выполнен методом обратной задачи кинематики. При этом передаточная функция регулятора:

$$W_{\text{р}}(s) = \frac{1}{T_{\text{ж}}\, s\, W_{\text{о}}(s)} = \frac{k_e\, T_{\text{м}}}{T_{\text{ж}}} + \frac{k_e}{T_{\text{ж}}} \cdot \frac{1}{s}$$

Откуда коэффициенты регулятора:

$$k_{\text{п}} = \frac{k_e\, T_{\text{м}}}{T_{\text{ж}}}, \qquad k_{\text{и}} = \frac{k_e}{T_{\text{ж}}}$$

В файле Init.c были записаны все переменные. Возьмем их из файла motor:

dt= 0.002

k_i = 0.16

k_p = 0.0064

pwm_period = 200

Tj = dt * 3=0.006

Tm = 0.0064 / 0.16 = 0.04

ke = k_i * Tj = 0.16*0.006 = 0.00096

В итоге в системе управления в основном цикле Cyclic.c представлены 3 функциональных блока с программой, подающей на систему управления ступенчатое воздействие.

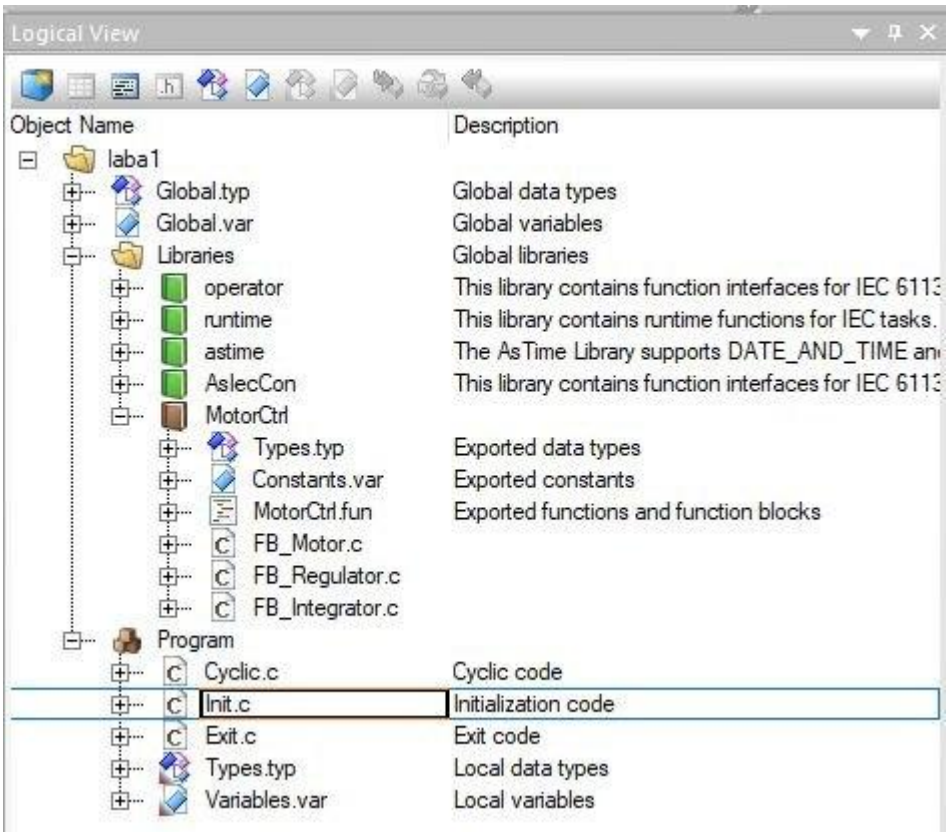


Рис. 8 - Параметры программы

С помощью инструмента Trace были получены графики входного воздействия, двигателя без регулятора и двигателя с регулятором



Рис. 9 - График уставки, скорости вращения двигателя без регулятора и скорости вращения двигателя с настроенным регулятором.

ВЫВОД

В ходе лабораторной работы были реализованы 3 функциональных блока, написана программа, совмещающая в себя созданные блоки. Были разработаны системы управления двигателем с регулятором и без. Регулятор позволяет контролировать некоторые параметры переходного процесса(время прихода в установившийся режим). Таким образом система с регулятором входит в установившийся режим за указанное нами время

Приложение А

Init.c

```
#include <bur/plctypes.h>
#ifdef _DEFAULT_INCLUDES
#include <AsDefault.h>
#endif
void _INIT ProgramInit(void)
{
    fb_motor.ke=0.00096;
    fb_motor.Tm=0.04;
    fb_motor.dt=0.002;
    fb_motor.w=0.0;
    fb_motor.phi=0.0;
    fb_motor.integrator.out=0.0;
    fb_motor_2.ke=0.00096;
    fb_motor_2.Tm=0.04;
    fb_motor_2.dt=0.002;
    fb_motor_2.w=0.0;
    fb_motor_2.phi=0.0;
    fb_motor_2.integrator.out=0.0;
    fb_controller.k_p=0.0064;
    fb_controller.k_i=0.16;
    fb_controller.max_abs_value=24.0;
    fb_controller.dt=0.002;
    fb_controller.u=0.0;
    fb_controller.iyOld=0.0;
    fb_controller.integrator.out=0.0;
    Enable=0;
    Speed=0.0;
    counter=0;
}
```

Приложение В

Cyclic.c

```
#include <bur/plctypes.h>
#ifdef _DEFAULT_INCLUDES
#include <AsDefault.h>
#endif
void _CYCLIC ProgramCyclic(void)
{
    USINT i;
    if (!Enable) {
        counter=0;
        Speed=0.0;
        return;
    }
    counter++;
    /* Cyclic #3 = 50 ms: 400 cycles = 20 s. */
    if ((counter >= 400) && (counter < 800))
        Speed=60.0;
    else
        Speed=0.0;
     /* 50 ms / 2 ms = 25 model calculation steps. */
    for (i=0; i<25; i++) {
        fb_controller.e=Speed-fb_motor.w;
        FB_Regulator(&fb_controller);
        fb_motor.u=fb_controller.u;
        FB_Motor(&fb_motor);
        fb_motor_2.u=Speed * fb_motor_2.ke;
FB_Motor(&fb_motor_2);
}
/* Full experiment: 60 seconds. */
if (counter >= 1200)
counter=0;
}
```

Приложение С

FB_Integrator.c

```
#include <bur/plctypes.h>
#include "Library.h"
void FB_Integrator(struct FB_Integrator* inst)
{
    inst->out=inst->out + inst->in * inst->dt;
}
```

Приложение D

FB_Motor.c

```
#include <bur/plctypes.h>
#include "Library.h"
void FB_Motor(struct FB_Motor* inst)
{
    REAL e;
    e=inst->u/inst->ke-inst->w;
    inst->integrator.in=e/inst->Tm;
    inst->integrator.dt=inst->dt;
    FB_Integrator(&inst->integrator);
    inst->w=inst->integrator.out;
    inst->phi=inst->phi   +   inst->w   *   0.1047197551   *
inst->dt;
}
```

Приложение E

FB_Regulator

```
#include <bur/plctypes.h>
#include "Library.h"
void FB_Regulator(struct FB_Regulator* inst)
{
    REAL p_term, sum_before_sat, u_sat;
    p_term=inst->e * inst->k_p;
    if (p_term > inst->max_abs_value)
    p_term=inst->max_abs_value;
    else if (p_term < -inst->max_abs_value)
    p_term=-inst->max_abs_value;
    inst->integrator.in=inst->e * inst->k_i + inst->iyOld;
    inst->integrator.dt=inst->dt;
    FB_Integrator(&inst->integrator);
    sum_before_sat=p_term + inst->integrator.out;
    if (sum_before_sat > inst->max_abs_value)
        u_sat=inst->max_abs_value;
```

```
    else if (sum_before_sat < -inst->max_abs_value)
        u_sat=-inst->max_abs_value;
    else
        u_sat=sum_before_sat;
    inst->iyOld=u_sat-sum_before_sat;
    inst->u=u_sat;
}
```